Developer Challenge Specifications

Joint Developer Challenge: Student Progress Tracker

⏱️ **Estimated Time to Complete:** 2–6 hours (per developer)

🎯 Project Goal

Build a working MVP of a simplified EdTech application that tracks and displays student progress data across multiple learning platforms. This simulates real-world scenarios where schools need unified visibility into student performance from various educational tools and vendors.

EdTech Context: In education technology, schools often use 10+ different learning platforms (reading apps, math tools, assessment systems, etc.). This challenge simulates the core problem we solve: aggregating student data from multiple sources into a unified dashboard that teachers and administrators can actually use.

🔗 Integration Plan

- Frontend fetches student data from the backend API using RESTful JSON exchange
- Backend provides secure, filtered access to student progress data
- If collaborating, coordinate on API contract design upfront
- If working separately, use the provided API specification and mock data

🛡️ Security & Data Handling Requirements

- Implement basic authentication simulation (mock JWT tokens or API keys)
- Include proper input validation and sanitization
- Handle sensitive student data appropriately (no real PII in demos)
- Implement role-based data filtering (teachers vs. administrators)

📊 Data Transformation Requirements

- Support filtering by grade level, subject, and date ranges
- Calculate and display aggregate statistics (class averages, progress trends)
- Handle missing or incomplete data gracefully
- Implement basic data pagination for large datasets

🚨 Error Handling & Edge Cases

- Network failures and API timeouts
- Invalid or corrupted data responses
- Empty datasets and missing student records
- Concurrent user access scenarios
- Rate limiting simulation

🤖 AI-Assisted Development Documentation

Required AI Usage Documentation:

- **AI Tools Used**: List all AI tools utilized (ChatGPT, Claude, GitHub Copilot, Cursor, etc.)
- **Prompt Evolution Log**: Document your actual prompts and how you refined them
- **Decision Rationale**: Explain when you chose AI assistance vs. manual coding
- **Critical Evaluation**: Describe instances where you rejected or modified AI suggestions
- **Learning Integration**: How you combined AI output with your domain knowledge

Documentation Format:

```None
## AI Development Log

### Tool: [AI Tool Name]
**Session 1: Initial Component Setup**
- Prompt: "Create a Vue 3 component for student progress table
with TypeScript"
- Result: [Brief description of output quality]
- Action Taken: [Used as-is/Modified/Rejected] and why
- Learning: [What this taught you about the problem/solution]

**Session 2: Problem Refinement**
- Problem: [Specific issue you encountered]
- Prompt: "How to handle pagination with large datasets in Vue
3..."
- Iteration: [How you refined the prompt based on initial
results]
- Final Solution: [Your implementation vs. AI suggestion]
```

🧠 Critical Thinking Assessment Areas

- **Problem Decomposition**: How you broke down complex requirements into AI-manageable chunks
- **Prompt Engineering**: Quality and progression of your prompts showing analytical thinking
- **Solution Evaluation**: Your ability to critically assess and improve AI-generated code
- **Integration Strategy**: How you combined multiple AI suggestions into cohesive solutions
- **Knowledge Application**: Evidence of applying domain expertise to guide AI interactions
- GitHub repo or ZIP file with complete source code
- Local setup instructions (including environment setup)
- Screenshots of UI showing different states (loaded, loading, empty, error)
- Brief README explaining architectural decisions and trade-offs
- **AI Development Log documenting tools, prompts, and critical thinking process**

Backend:

- GitHub repo or ZIP file with complete .NET solution
- Functional REST API with comprehensive endpoints
- Database schema/migrations with seed data (minimum 20 diverse records)
- API documentation (Swagger/OpenAPI or Postman collection)
- Brief README explaining security considerations and scalability approaches
- **AI Development Log documenting tools, prompts, and critical thinking process**

Both:

- Documentation of how you would integrate this with enterprise systems
- Notes on compliance considerations for student data (FERPA, privacy)
- **Comprehensive test suite with coverage reports and testing strategy documentation**
- **QA artifacts demonstrating systematic quality assurance approach**
- **AI Development Log showcasing critical thinking and problem-solving methodology**

Frontend Developer Challenge: Student Progress Tracker

⏱️ **Estimated Time to Complete:** 2–6 hours

🎯 Project Goal

Build a working MVP of a simplified EdTech application that tracks and displays student progress data across multiple learning platforms. This dashboard helps teachers and administrators understand how students are performing across different educational tools in real-time.

📱 Responsibilities

Tech Stack:

- Vue 3 + Composition API
- Tailwind CSS
- TypeScript
- Vite

Core Features:

1. **Responsive Student Progress Dashboard:**
    - Table/card view of students (name, grade, overall progress %, last activity, status)
    - Real-time progress indicators and trend arrows
    - Filter/search by name, grade level, subject area, and date range
    - Sort by various metrics (progress, last activity, alphabetical)
2. **Student Detail View:**
    - Modal or drawer with detailed progress breakdown
    - Subject-specific performance charts
    - Activity timeline and engagement metrics
    - Assignment completion status and upcoming deadlines
3. **Data Integration & State Management:**
    - Fetch student data from REST API endpoints
    - Implement proper loading states with skeleton screens
    - Handle API errors with user-friendly messages and retry options
    - Cache data appropriately to reduce API calls
    - Real-time updates simulation (polling or WebSocket mock)
4. **Advanced UI/UX Features:**

    - Role-based views (teacher vs. administrator perspectives)
    - Bulk actions (select multiple students for actions)
    - Export functionality (CSV/PDF simulation)
    - Dark/light theme toggle
    - Mobile-responsive design with touch-friendly interactions

🛡️ Security & Data Requirements

- Implement mock authentication (login screen with role selection)
- Display appropriate data based on user role (teachers see their classes only)
- Sanitize all user inputs and search queries
- Handle sensitive student data with appropriate privacy considerations

🚨 Error Handling & Edge Cases

- Network connectivity issues
- API rate limiting and timeout scenarios
- Empty search results and no data states
- Invalid user inputs and edge case validation
- Graceful degradation when features are unavailable

📊 Performance & Accessibility

- Optimize for large datasets (100+ students)
- Implement virtual scrolling or pagination for performance
- Ensure WCAG 2.1 AA compliance (semantic HTML, ARIA labels, keyboard navigation)
- Support screen readers and high contrast modes
- Minimize bundle size and optimize loading performance

📦 Deliverables

- GitHub repo or ZIP file with complete Vue 3 application
- Local setup instructions with dependency management
- Screenshots showing different UI states and responsive behavior
- README documenting:
    - Component architecture and reusability approach
    - Performance optimization strategies
    - Accessibility implementation details
    - Future enhancement possibilities
    - **AI tool usage and prompt engineering methodology**

🧪 Testing & Quality Assurance Requirements

- **Unit Testing**: Implement comprehensive unit tests with >70% code coverage

- **Component Testing**: Test Vue components in isolation with Vue Test Utils
- **Integration Testing**: Test API integration and data flow end-to-end
- **Accessibility Testing**: Automated accessibility testing with axe-core
- **Error Boundary Testing**: Verify graceful error handling and recovery
- **Performance Testing**: Lighthouse audits and bundle size optimization

🔍 QA Deliverables

- Test suite with clear test organization and naming conventions
- Screenshot/visual regression tests for critical UI components
- Documentation of testing strategy and coverage reports
- Bug report template demonstrating QA thinking
- Performance audit results and optimization recommendations

🎯 Optional Extensions (Bonus Points)

- Add basic data visualization (charts showing progress trends)
- Implement offline functionality with service workers
- Add real-time notifications for student milestones
- Create a simple admin panel for user management
- End-to-end testing with Cypress or Playwright

Backend Developer Challenge: Student Progress Tracker

⏱️ **Estimated Time to Complete:** 2–6 hours

🎯 Project Goal

Build a robust REST API for an EdTech student progress tracking system. This backend simulates the data integration challenges schools face when consolidating information from multiple learning platforms into a unified system.

⚙️ Responsibilities

Tech Stack:

- .NET Core (latest LTS)
- Entity Framework Core
- SQL Server/SQLite
- RESTful API design
- Swagger/OpenAPI documentation

Core API Endpoints:

1. **Student Management API:**

```
None
GET /api/students - List students with filtering and pagination
GET /api/students/{id} - Get detailed student information
GET /api/students/{id}/progress - Get student progress metrics
POST /api/students/{id}/progress - Update student progress data
```

2. **Data Filtering & Search:**
    - Support query parameters: grade, subject, dateRange, searchTerm
    - Implement efficient pagination with total count
    - Provide sorting options (name, progress, lastActivity)
    - Include metadata for frontend state management
3. **Authentication & Authorization API:**

```
None
POST /api/auth/login - Mock authentication endpoint
GET /api/auth/profile - Get current user profile and permissions
GET /api/users/{id}/students - Get students assigned to specific
teacher
```

4. **Analytics & Reporting:**

```
None
GET /api/analytics/class-summary - Class-level statistics
GET /api/analytics/progress-trends - Historical progress data
GET /api/reports/student-export - Export student data (CSV
format)
```

🏗️ Data Architecture & Performance

- Design normalized database schema with proper relationships
- Implement Entity Framework migrations with comprehensive seed data
- Create efficient database queries with proper indexing considerations
- Add query optimization for large datasets (1000+ students)
- Implement basic caching strategy for frequently accessed data

🛡️ Security & Validation

- Input validation with comprehensive error responses
- SQL injection prevention through parameterized queries
- Implement role-based access control (RBAC) simulation
- Data sanitization and proper error handling
- Rate limiting simulation and API throttling considerations

🚨 Error Handling & Resilience

- Global exception handling with structured error responses
- Proper HTTP status codes and error messaging
- Database connection failure handling
- Validation error aggregation and user-friendly messages
- Logging strategy for debugging and monitoring

🔄 Integration & Scalability Considerations

- Design for horizontal scaling (stateless operations)
- Implement async/await patterns for I/O operations
- Create modular service architecture for future extensions
- Design API versioning strategy
- Consider event-driven patterns for real-time updates

📦 Deliverables

- GitHub repo or ZIP file with complete .NET solution
- Comprehensive API documentation (Swagger/OpenAPI)
- Database schema documentation and migration scripts
- README including:
    - Architecture decisions and patterns used
    - Security implementation details
    - Performance optimization strategies
    - Integration considerations for enterprise environments
    - Scalability and deployment notes
    - **AI tool usage and prompt engineering methodology**

🧪 Testing & Quality Assurance Requirements

- **Unit Testing**: Comprehensive unit tests with >80% code coverage using xUnit/NUnit
- **Integration Testing**: Test API endpoints, database operations, and service interactions
- **Contract Testing**: Ensure API contracts match frontend expectations
- **Performance Testing**: Load testing for concurrent users and large datasets
- **Security Testing**: Input validation, SQL injection prevention, authentication flows
- **Database Testing**: Migration testing, seed data validation, query optimization

🔍 QA Deliverables

- Automated test suite with clear test organization and documentation
- Postman collection with comprehensive API test scenarios
- Performance test results with bottleneck analysis
- Security test report demonstrating input validation and data protection
- Database migration and rollback testing documentation
- Mock data generation strategy for consistent testing

🎯 Optional Extensions (Bonus Points)

- Implement basic caching with Redis simulation
- Add database query performance monitoring
- Create automated API tests with sample data

- Implement basic audit logging for data changes
- Add health check endpoints for monitoring
- Integration testing with test containers (Testcontainers.NET)

📊 Sample Data Requirements

Minimum 20 diverse student records including:

- Students across grades K-12
- Multiple subjects (Math, Reading, Science, Social Studies)
- Various progress levels (struggling, on-track, advanced)
- Different activity patterns (daily active, sporadic, inactive)
- Realistic timeline data spanning several months

Progress Metrics to Track:

- Overall completion percentage
- Subject-specific performance scores
- Time spent in learning activities
- Assignment completion rates
- Assessment scores and trends
- Last activity timestamps

Evaluation Focus Areas

For Both Challenges:

1. **Code Quality:** Clean, maintainable, well-organized code
2. **Documentation:** Clear setup instructions and architectural explanations
3. **Error Handling:** Comprehensive edge case management
4. **Security Awareness:** Proper data handling and validation
5. **Scalability Thinking:** Considerations for real-world deployment
6. **Testing Strategy:** Comprehensive test coverage and QA approach
7. **Quality Assurance:** Systematic approach to bug prevention and detection

Frontend Specific:

- Component reusability and Vue 3 best practices
- TypeScript implementation quality
- Responsive design and accessibility
- Performance optimization
- **Testing coverage and component testing approach**

- **QA methodology for UI/UX validation**

Backend Specific:

- API design and REST principles
- Database design and query optimization
- Authentication and authorization patterns
- Integration architecture considerations
- **Unit and integration testing comprehensiveness**
- **Performance and security testing implementation**

Integration Assessment:

- API contract design and communication
- Error handling across system boundaries
- Data consistency and validation
- Documentation of integration points
- **End-to-end testing strategy**
- **System-level QA and monitoring approach**